書籍管理システム（Global Books Vue3-Django）- スターターキット

これは、Vue.js 3とDjango REST Frameworkを使用して書籍管理システムを構築するためのスターターキットです。 基本的なプロジェクト構成や設定は完了していますが、主要な機能は未実装です。 以下の手順に従って、開発を進めてください。

ZIPの受け取りとVS Codeの準備

以下は、受講者のPCで `C:\work` を作業場所にする例です。

1. **講師からZIPファイルを受け取る**

   配布された `globalbooks_starter_v1.1.0.zip` を自分のPCに保存します。

2. **ZIPを展開する**

   エクスプローラーで `C:\work` フォルダを作成します。ZIPを右クリックして「すべて展開」を選び、展開先を `C:\work` に指定します。

   展開後の作業フォルダは、次の場所です。

   ```
   C:\work\globalbooks_Vue3-Django_starter
   ```

3. **VS Codeで作業フォルダを開く**

   VS Codeの「ファイル」→「フォルダーを開く」から、上記のフォルダを選びます。`backend-django`・`frontend`・`docs`・`README.pdf` が同じ階層に見えるフォルダが正解です。展開時にフォルダが二重になった場合は、その内側を開いてください。

   ```
   globalbooks_Vue3-Django_starter  ← このフォルダを開く
   ├─ backend-django
   ├─ frontend
   ├─ docs
   └─ README.pdf
   ```

4. **ターミナルを開いて場所を確認する**

   VS Codeの「ターミナル」→「新しいターミナル」を開き、PowerShellで次を実行します。

   ```
   cd C:\work\globalbooks_Vue3-Django_starter
   pwd
   dir
   ```

   `pwd` の表示が作業フォルダになり、`dir` で上記のファイル・フォルダが確認できれば準備完了です。別の場所へ展開した場合は、以降の `C:\work\globalbooks_Vue3-Django_starter` を自分の作業フォルダに読み替えます。

チーム開発のルール

この演習は3〜4名のチームで開発します。実装を始める前に、誰がどの部分を担当するか、全員で話し合って決めてください。

設計書を確認し、画面・APIの担当、共通ファイルを変更する担当、結合作業の担当を決めます。同じファイルを複数人が変更する場合は、あらかじめ変更箇所を相談してください。

GitやSubversionなどの共同開発ツールは、基本的に提供しません。講師が用意した共有フォルダを使って、ソースコードを受け渡し・結合します。

作業場所	用途
各自のPCの作業フォルダ	開発・サーバー起動・動作確認を行う
共有フォルダ内の各自用フォルダ	結合するソースコードをアップロードする
共有フォルダ内のチーム代表ソース用フォルダ	結合済みの、チームとしての最終ソースコードを格納する

共有フォルダには、各自用フォルダとチーム代表ソース用フォルダが作成されています。実際の場所・フォルダ名は講師の案内を確認してください。

開発・結合の流れ

1. **各自のPCで開発する**

   チームで担当を決めた後、全員が以下の「セットアップ & 開発手順」で自分の環境を準備します。日々の編集・動作確認はローカルで行います。

2. **各自用の共有フォルダへアップロードする**

   結合する際に、変更したソースコードを自分の共有フォルダへコピーします。元のフォルダ構成を保ち、変更したファイルと内容をチームに伝えてください。

3. **代表ソースへファイルを結合する**

   結合担当が共有フォルダ上の各メンバーの変更を確認し、チーム代表ソースへ反映します。同じファイルの変更は内容を見比べてまとめ、他の人の変更を消さないようにします。

4. **ローカルで全体を確認し、代表ソースを更新する**

   結合したソースをPCへコピーし、全体が動くか確認します。修正した場合は、確認済みの内容を代表ソース用フォルダへ戻します。各メンバーも最新の代表ソースをローカルへ取り込み、次の開発に進みます。

`node_modules`・`.venv`・`dist`・`__pycache__` は受け渡し不要です。`db.sqlite3` は各自で作成し、ソースコードの結合時には上書きしません。依存パッケージやDB定義を変更した場合は、必要なインストール・マイグレーション手順もチームへ伝えてください。

機能要件（実装目標）

- 書籍一覧表示（画像付き）
- 書籍詳細表示
- 新規書籍登録
- 書籍編集
- 書籍削除

技術スタック

- **Frontend**: Vue.js 3, Vite, Bootstrap 5
- **Backend**: Django 5, Django REST Framework
- **Database**: SQLite3

セットアップ & 開発手順

前提条件

- Python 3.10+（演習環境は3.13）
- Node.js 20.19以上の20系、または22.12以上（今回のVite 7に合わせて変更）
- SQLite3コマンドラインツール（`sqlite3` コマンドを実行できること）

追記： 演習で使うPythonパッケージを、最初に1回インストールしてください。

```
python -m pip install "Django>=5.2.10,<5.3" "djangorestframework==3.16.1" "django-cors-headers==4.9.0"
```

1. バックエンド (Django) のセットアップ

バックエンド用のDjangoディレクトリと、DB作成用SQLだけ準備してあります。 以下の手順に従いセットアップを行ってください。

1. **ディレクトリ移動**

   このREADMEのあるフォルダから実行します。

```
cd backend-django
```

2. **Djangoプロジェクトの作成**

   Djangoプロジェクトを作成します。

```
django-admin startproject config .
```

3. **アプリケーションの作成**

   書籍管理用のアプリ `books` を作成します。

```
python manage.py startapp books
```

4. **ディレクトリ構成 (Backend)**

   以下のディレクトリ構成になっていることを確認してください。 （実行結果は一部省略して表示しています）

```
tree /f ./
```

```
backend-django
│  initial_data.sql
│  manage.py
│
├─books
│  │  models.py
│  │  views.py
│  │
│  └─migrations
│
└─config
   │  settings.py
   └─ urls.py
```

5. **設定の追加**

`config/settings.py` に必要な情報を追加してください。

- `INSTALLED_APPS` にアプリを追加

```
'rest_framework',  # 追加: REST API用
'corsheaders',     # 追加: CORS用
'books',           # 新規: 書籍管理アプリ
```

- `MIDDLEWARE` にCORSの設定を先頭に追加

```
'corsheaders.middleware.CorsMiddleware',
```

- `CORS_ALLOWED_ORIGINS` を定義して、オリジンサーバを追加

```
CORS_ALLOWED_ORIGINS = [
    'http://localhost:5173',
    'http://localhost:5174',
    'http://localhost:5175',
]
```

6. **モデルの作成**

テーブル定義を基にモデルを作成してください。以下に注意して作成します。

- ID列は自動生成とし、フィールド定義は不要です。
- nullableとはNullを許可する設定で、`null=True, blank=True` の設定で実現します。
- テーブル名は `books` とします。アプリケーション名がつかないように注意してください。モデル内の `class Meta` に `db_table = 'books'` を指定します。（Djangoのデフォルトは「アプリ名_モデル名」で生成）

追記（追加機能用の列）： `Book` モデル内に、次の1行も追加してください。後から列を追加する作業を避けるため、最初から用意します。基本機能では使わず、初期値0のまま保持します。

```
like_count = models.PositiveIntegerField(default=0)
```

7. **データベースのマイグレーション**

   モデルを作成した後に実行します。

```
python manage.py makemigrations
python manage.py migrate
```

8. **初期データの投入（SQL）**

   `backend-django` フォルダに `initial_data.sql` が用意されています。 SQLite3コマンドラインツールでデータを投入してください。 データ投入後、テーブルのレコードが表示されるため、確認してください。

```
sqlite3 db.sqlite3
.read initial_data.sql
.quit
```

   追記： 空の書籍テーブルに初回だけ投入します。`.read` と `.quit` はSQLiteのプロンプト内で実行し、10件の表示を確認してください。

9. **サーバー起動**

```
python manage.py runserver
```

   標準ポート8000で起動します。

2. フロントエンド（Vue）のセットアップ

フロントエンドは、必要な画面やコンポーネントのファイル（枠組み）のみ用意されています。

1. **ディレクトリ移動とインストール**

   Djangoのサーバーは起動したままにします。VS Codeのターミナル右上の「＋」で別のターミナルを開き、作業フォルダへ移動してから実行します。

```
cd C:\work\globalbooks_Vue3-Django_starter
cd frontend
npm install
```

2. **開発サーバー起動**

```
npm run dev
```

3. **ブラウザで確認**

   アクセス: http://localhost:5173

   画面に表示される指示（コメント）に従って、実装を進めてください。

補足

・ APIの仕様や画面設計については、別途配布される設計書を確認してください。

追記：今回の追加機能（任意）

基本機能を完成させた後、次の3つに任意で取り組めます。

・ 並び替えボタンの追加

・ 書籍ごとのいいね

・ 画像ファイル添付

いいね用の列は手順6で用意済みです。APIの実装時には、`BookSerializer` の `class Meta` に `read_only_fields = ['like_count']` を指定し、通常の登録・編集で回数を上書きしないようにします。

画像ファイル添付に取り組む場合だけ、Pillowを追加でインストールしてください。

```
python -m pip install "Pillow>=11.1,<13"
```

画面例は同梱の基本設計書の第7章「追加機能」を確認してください。